SMART DEADLOCK DETECTION SIMULATOR

An Innovative Assignment for the subject of Operating System

Submitted by:

Name: Trishna Das

Registration no.: 2302041055

Semester: 5th

Subject name: Operating System (OS)

Department Of Computer Science & Engineering

Veer Surendra Sai University Of Technology, Burla

Submitted To :

Prof. Himansu Sekhar Behera

Professor & Head of Department,

Department Of Computer Science & Engineering,

VSSUT, Burla

TABLE OF CONTENTS

SMART DEADLOCK DETECTION SIMULATOR.....1
1.1 Abstract.....2
1.2 Introduction.....2
2. Objectives.....3
3. Methodology.....4
Core Algorithms.....4
System Architecture and Design.....5
Technology Stack and Implementation Details..... 6
System Workflow and Modules.....7
4. Results.....8
User Interface and Simulation Scenarios..... 8
Scenario 1: Deadlock Detected..... 8
Scenario 2: No Deadlock.....9
Scenario 3: Deadlock Avoidance (Banker's Algorithm).....10
5. Conclusion..... 11
6. Future Scope.....12
7. References.....13

1.1 Abstract

Deadlock is a critical failure state in concurrent operating systems, where a set of processes becomes permanently blocked, each waiting for a resource held by another process in the set. This project report details the design and implementation of a smart, interactive simulator developed to elucidate the complex mechanisms of deadlock. The system provides a web-based graphical interface for visualizing the interactions between processes and resources in real-time. The main goal is to create a powerful educational tool that successfully demonstrates the conditions that lead to deadlock and the logical rigor of the algorithms designed to manage it, thereby making abstract operating system concepts tangible and comprehensible.

1.2 Introduction

In modern operating systems, concurrency allows multiple processes to execute simultaneously and share resources to maximize efficiency. However, this introduces complex challenges, the most severe being deadlock—a state where two or more processes are unable to proceed because each is waiting for a resource held by another. This results in a permanent standstill that can degrade performance or cause a complete system hang.

While foundational algorithms like the Wait-For Graph for detection or the Banker's Algorithm for avoidance provide theoretical solutions, their operational logic can be difficult to grasp from static text and diagrams alone. This simulator was conceived to bridge the significant gap between abstract theory and practical understanding. It transforms raw, numerical data from resource allocation and request matrices into a dynamic and interactive visual demonstration, vividly illustrating the formation of a deadlock cycle. This interactive approach demystifies one of the most challenging topics in concurrent programming and operating system design.

2. Objectives

- **1. To Design and Implement a Simple and Efficient Simulation Model:**
  The main goal was to create a lightweight simulation that accurately models the interaction between processes and resources within an operating system. The emphasis was on efficiency and responsiveness, ensuring that the simulator runs smoothly and reacts to user input in real time. The system follows an object-oriented approach, using classes such as *Process*, *Resource*, and a central *ResourceManager* to mimic real OS behavior in a controlled environment.
- **2. To Visually Demonstrate Deadlock Formation and Resolution:**
  Another major objective was to translate the abstract concept of deadlocks into an intuitive graphical representation. The simulator visualizes resource allocation and waiting conditions as a dynamic graph, where circular dependencies directly indicate deadlocks. Color-coding (e.g., red for blocked processes) highlights critical states, allowing users to clearly observe how and why a deadlock occurs.
- **3. To Implement and Compare Smart Deadlock Handling Algorithms:**
  The project aimed to implement two classical approaches—**Deadlock Detection** and **Deadlock Avoidance**. Detection is achieved using a Wait-for Graph with cycle detection (DFS), showing how the system identifies existing deadlocks. Avoidance is demonstrated using the **Banker's Algorithm**, which proactively prevents unsafe states. This contrast helps learners understand the difference between reactive and preventive strategies.
- **4. To Enable Interactive User-Controlled Analysis:**
  A key objective was to make the system interactive, allowing users to define the number of processes, resources, and allocation matrices. Users can trigger the algorithms manually to observe different outcomes, perform "what-if" experiments, and instantly visualize how small changes affect the system's state. This hands-on exploration deepens conceptual understanding through active learning.

3. Methodology

Core Algorithms

The simulator's engine is built on three fundamental algorithms that model how operating systems detect, avoid, and recover from deadlocks. These algorithms together form the foundation for understanding process synchronization and resource management.

1. Deadlock Detection – Wait-for Graph (WFG):
For systems where each resource type has a single instance, deadlock detection is based on a **Wait-for Graph (WFG)**. In this directed graph, nodes represent processes, and an edge from *Pi* → *Pj* indicates that *Pi* is waiting for a resource held by *Pj*. A deadlock exists if and only if the graph contains a cycle. The simulator constructs the WFG dynamically and applies a **Depth-First Search (DFS)** cycle-detection algorithm to identify such circular waits. When a cycle is found, the involved processes are marked as deadlocked.

2. Deadlock Avoidance – Banker's Algorithm:
For multiple instances of resources, the simulator uses the **Banker's Algorithm**, which proactively prevents deadlocks by ensuring that the system always remains in a safe state. Each process declares its **maximum resource demand** beforehand. The algorithm maintains four key data structures:

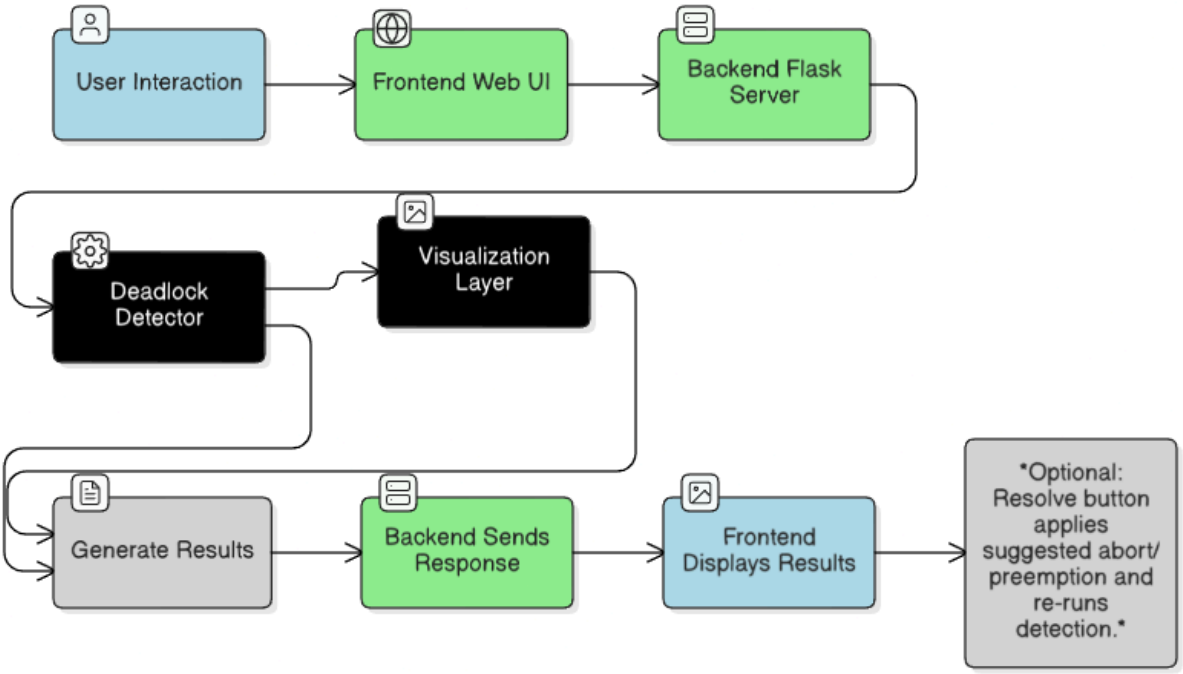
- *Available*: Instances of each resource currently free.
- *Max*: Maximum demand of each process.
- *Allocation*: Resources currently held by each process.
- *Need*: Remaining resources required (Need = Max – Allocation).
  Two core routines are implemented — the **Safety Algorithm**, which checks if the system state is safe, and the **Resource-Request Algorithm**, which grants a request only if it leaves the system in a safe state.

3. Smart Suggestion Algorithm (Recovery):
When a deadlock is detected, the simulator applies a heuristic-based recovery mechanism. It identifies all processes in the deadlock cycle and suggests terminating the one holding the fewest resources. This minimizes disruption while releasing enough resources to resolve the deadlock efficiently. The suggestion is displayed to the user alongside the detected cycle for better understanding of system recovery.

System Architecture and Design

The simulator is built on a client-server model, which cleanly separates the user interface from the core deadlock detection logic.

- **1. Client-Server Model:**
  The simulator follows a client-server design, separating the user interface from the backend deadlock detection logic.
- **2. Simulator Core Engine (Discrete-Event Model):**
  Built as a discrete-event simulation where system states change only on specific events like resource requests or releases. Uses object-oriented design with Process, Resource, and ResourceManager classes.
- **3. Algorithmic Integration:**
  The ResourceManager supports two modes — **Detection Mode** (Wait-For Graph) and **Avoidance Mode** (Banker's Algorithm).
- **4. Real-Time Visualization:**
  Uses **WebSockets** for live updates between backend and frontend. Implements the **Observer pattern** where ResourceManager acts as Subject and WebSocket server as Observer, ensuring real-time synchronized visualization.



System workflow

Technology Stack and Implementations

Component	Technology
Backend	Python (Flask Framework)
Frontend	HTML, CSS, JavaScript (Bootstrap for styling)
Visualization	NetworkX + Matplotlib (Python Graph Libraries)
Deployment	Render / PythonAnywhere / localhost
Programming Paradigm	Object-Oriented Python with REST APIs

Backend (Python with Flask): Python was chosen for its expressive syntax and strong support for data structures. The Flask framework provides the server-side application logic.

A conceptual implementation of the Safety Algorithm demonstrates the core logic:

```
# Conceptual implementation of the Safety Algorithm
def is_safe_state(self, available, allocation, need):
  num_processes = len(allocation)
  work = list(available)
  finish = [False] * num_processes
  while True:
    found_process = False
    for i in range(num_processes):
      if not finish[i] and all(need[i][j] <= work[j] for j in range(len(work))):
        for j in range(len(work)):
          work[j] += allocation[i][j]
        finish[i] = True
        found_process = True
    if not found_process:
      break
  return all(finish)
```

Frontend and Visualization: The frontend uses standard HTML, CSS, and JavaScript. The visualization is generated on the backend using **NetworkX** to construct the graph and **Matplotlib** to render it as a color-coded image, which is then sent to the frontend.

System Workflow and Modules

The project is structured into logical modules:

- **app.py**: The main Flask application file that handles API routes and renders HTML templates.
- **deadlock_detector.py**: Contains the core logic for graph creation and cycle detection.
- **templates/**: Contains the HTML files for the user input and result display.
- **static/**: Holds all static assets like CSS and JavaScript files.
- **visualizer.py**: Generates the visual graph plots from the Wait-For Graph.

The workflow is as follows:

1. **Input Phase**: The user enters the number of processes and resources and provides the Allocation and Request matrices on the web interface.
2. **Graph Construction**: The backend receives this data and constructs a Wait-For Graph (WFG).
3. **Detection Algorithm**: A DFS-based cycle detection algorithm checks for a circular wait in the WFG.
4. **Output & Visualization**: The Flask app sends the result (Deadlock Detected / No Deadlock) and the color-coded graph back to the web page, along with a smart suggestion for recovery if a deadlock exists.

4. Results

User Interface and Simulation Scenarios

Interface Design:
The simulator interface is structured for clarity and interactivity, featuring a **visualization panel**, a **control panel for inputs**, and a **log area** for textual feedback. Processes appear as circular nodes — **green for active**, **red for blocked**, and arrows indicate waiting dependencies.

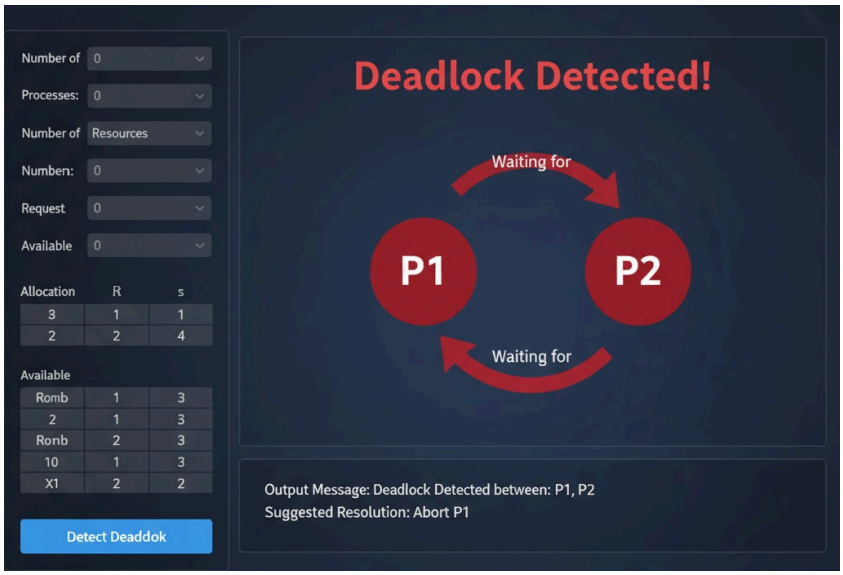
Scenario 1: Deadlock Detected

Setup: Two processes (P1, P2) and two single-instance resources (R1, R2).
Progression:

- P1 → R1 (granted)
- P2 → R2 (granted)
- P1 requests R2 → blocked (R2 held by P2)
- P2 requests R1 → blocked (R1 held by P1)

Detection & Output:
The user clicks *Detect Deadlock*. The backend constructs a **Wait-for Graph**, detects a cycle via **DFS**, and reports:

- **Output:** "Deadlock Detected between P1, P2"
- **Suggested Resolution:** Abort P1
- **Visualization:** Red nodes (P1, P2) connected in a cycle.



Scenario 2: No Deadlock

Setup: The system is configured without circular dependencies.

Output: “No Deadlock Found.”

Visualization: All process nodes are **green**, showing a safe, runnable state.

Scenario 3: Deadlock Avoidance (Banker’s Algorithm)

Setup: Multiple resource instances (e.g., 10 total), three processes with defined Max and Allocation matrices.

Unsafe Request: A process requests 2 instances (3 available). The **Safety Algorithm** detects no safe sequence → request **denied**.

Safe Request: A smaller request leads to a safe sequence → request **granted**.

Output:

- “Request DENIED – Unsafe State” or
- “Request GRANTED – Safe State.”

Visualization: Updates dynamically to reflect new allocations.

5. Conclusion

This project successfully culminated in the design and implementation of an interactive simulator for deadlock detection and avoidance in operating systems. By marrying robust algorithmic backends with a dynamic web-based visualization, the simulator transcends abstract theoretical concepts, offering a tangible and intuitive learning experience.

The system effectively demonstrates the intricate conditions leading to deadlock and the proactive or reactive strategies for its management, notably through the Wait-for Graph and Banker's Algorithm. The choice of a Python Flask backend for algorithmic heavy lifting and a JavaScript-driven frontend for real-time interaction proved instrumental in achieving a responsive and pedagogically effective tool.

Ultimately, this simulator stands as a testament to the power of visualization in demystifying complex computer science principles, providing a clear, insightful, and practical understanding of resource contention and deadlock resolution in concurrent environments.

6. Future Scope

- **Enhanced Deadlock Recovery Strategies:** Beyond simple process termination, future iterations could incorporate more sophisticated recovery mechanisms. This includes detailed visualization of resource preemption, process rollback to a safe state, and selection of optimal victim processes based on criteria like minimum cost (e.g., least CPU time used, fewest resources held), offering a more nuanced understanding of recovery trade-offs.
- **Integration of Dynamic Resource Management:** Currently, resource types and quantities are statically defined. A valuable extension would be to allow dynamic addition or removal of resources and processes during runtime, reflecting more realistic operating system behavior. This would enable users to observe the immediate impact of system changes on deadlock potential.
- **Advanced Visualization and Real-time Interaction:** Transitioning from static graph images to a fully interactive, client-side graph visualization (e.g., using D3.js, React Flow, or Vis.js) would unlock continuous, animated state transitions. This would allow users to visually trace resource requests and allocations step-by-step, highlighting how wait-for dependencies form and dissolve in real-time, greatly enhancing the intuitive understanding of algorithmic flow.
- **Multi-Instance Resource Allocation Graph (RAG) Visualization:** The current detection primarily focuses on the Wait-for Graph (single-instance resources). Expanding the visualization to include true Resource Allocation Graphs, distinguishing between request and assignment edges for multiple instances of resources, would provide a more comprehensive view applicable to the Banker's Algorithm and general resource management.
- **Formal Verification and Performance Benchmarking Module:** For more advanced study, a module could be added to formally verify the safety algorithm's output or to benchmark the performance of different detection/avoidance strategies under varying loads and system configurations, providing quantitative insights into their efficiency and scalability.

7. References

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). John Wiley & Sons. (Textbook reference)
2. Tanenbaum, A. S., & Bos, H. (2015). Modern Operating Systems (4th ed.). Pearson. (Textbook reference)
3. GeeksforGeeks. Deadlock in Operating System. Retrieved from https://www.geeksforgeeks.org/deadlock-in-operating-system/
4. Tutorialspoint. Operating System - Deadlock. Retrieved from https://www.tutorialspoint.com/operating_system/os_deadlock.htm
5. GFG. Deadlock Prevention in Operating System. GeeksforGeeks. Retrieved from https://www.geeksforgeeks.org/deadlock-prevention-in-operating-system/
6. Bell, J. Operating Systems Course Notes: 7_Deadlocks. University of Illinois Chicago. Retrieved from https://www.cs.uic.edu/~jbell/CourseNotes/OperatingSystems/7_Deadlocks.html
7. Wikipedia. (2023, May). Wait-for graph. Retrieved from https://en.wikipedia.org/wiki/Wait-for_graph
8. Mishra, J. (2024, January 10). Deadlock Detection and Recovery. Retrieved from https://blog.jyotiprakash.org/deadlock-detection-and-recovery
9. GeeksforGeeks. Wait For Graph Deadlock Detection in Distributed System. Retrieved from https://www.geeksforgeeks.org/wait-for-graph-deadlock-detection-in-distributed-system/
10. Emory University. Detecting deadlocks using wait-for graphs. Retrieved from http://www.mathcs.emory.edu/~cheung/Courses/455/Syllabus/7-deadlock/wait-for-graph.html
11. Wikipedia. Banker's algorithm. Retrieved from https://en.wikipedia.org/wiki/Banker%27s_algorithm
12. GeeksforGeeks. Banker’s Algorithm in Operating System. Retrieved from https://www.geeksforgeeks.org/bankers-algorithm-in-operating-system/